

Proyecto Business Intelligence para Concesionaria de Vehículos

Integrantes Grupo 3

- Beatriz Eunice Beltrán López
 - Brenda Nicole Henríquez Amaya
 - Jonathan Vladimir Montes Rodríguez
-

Introducción

En este proyecto se desarrolló una solución de Business Intelligence para una concesionaria de vehículos, utilizando un sistema transaccional OLTP y un Data Warehouse OLAP.

El objetivo principal fue transformar datos operativos en información estratégica que ayude a la gerencia a tomar mejores decisiones relacionadas con ventas, clientes, inventario y servicios de taller.

Para lograrlo, se implementó un proceso ETL que permitió extraer, transformar y cargar la información desde la base de datos operacional hacia un entorno analítico optimizado para generar indicadores y reportes empresariales.

Además, se desarrollaron análisis de inteligencia de negocio y dashboards interactivos que permiten visualizar métricas importantes para la empresa.

Explicación de OLTP y OLAP

Sistema OLTP

El sistema OLTP (Online Transaction Processing) es el encargado de registrar las operaciones diarias de la concesionaria.

En este sistema se almacenan actividades como:

- Registro de clientes
- Ventas de vehículos
- Leads comerciales
- Inventario
- Órdenes de reparación
- Información de empleados

Su principal objetivo es manejar transacciones rápidas y precisas en tiempo real.

Sistema OLAP

El sistema OLAP (Online Analytical Processing) se utiliza para analizar la información del negocio.

A diferencia del OLTP, el OLAP está diseñado para:

- Analizar tendencias
- Generar reportes
- Crear KPIs
- Identificar patrones
- Ayudar en la toma de decisiones

Este sistema utiliza un Data Warehouse optimizado para consultas analíticas y visualización empresarial.

Relación entre ambos sistemas

El sistema OLTP almacena la información operativa diaria.

Posteriormente, mediante un proceso ETL:

1. Se extraen los datos del OLTP.
2. Se transforman y organizan.
3. Se cargan en el sistema OLAP.

Gracias a esto, la gerencia puede obtener información estratégica del negocio.

Modelo Operacional OLTP

El sistema OLTP fue diseñado para representar las operaciones principales de una concesionaria automotriz.

Las tablas implementadas fueron:

- empleados
- clientes
- vehiculos
- leads
- ventas

- ordenes_reparacion

Este modelo permite registrar la información diaria relacionada con ventas, inventario y servicios de taller.

A continuación se muestra el script SQL utilizado para crear la base de datos operacional.

Generación de Datos Operacionales

Para simular un entorno real de trabajo se desarrolló un script en Python encargado de generar información ficticia utilizando la librería Faker.

El sistema genera automáticamente:

- Empleados
- Clientes
- Vehículos
- Leads
- Ventas
- Órdenes de reparación

También se aplicó lógica de negocio para representar escenarios reales de una concesionaria, como:

- Vehículos vendidos
- Clientes premium
- Leads perdidos
- Vehículos que regresan al taller
- Márgenes de ganancia
- Créditos y garantías

Finalmente, todos los datos fueron cargados automáticamente en MySQL.

```
In [ ]: # -*- coding: utf-8 -*-
import pandas as pd
import random
from faker import Faker
from datetime import timedelta
from sqlalchemy import create_engine

# Inicializar Faker
fake = Faker('es_MX') # Genera datos en español
Faker.seed(42)
random.seed(42)

# =====
# 1. CONEXIÓN A MYSQL
# =====
```

```

# Cambia 'usuario' y 'password' por tus credenciales locales de MySQL
# Formato: mysql+pymysql://usuario:contraseña@localhost:3306/nombre_bd
engine = create_engine("mysql+pymysql://root:root@localhost:3306/concesionaria_oltp

print("Conexión exitosa a MySQL. Iniciando generación de datos...")

# =====
# 2. GENERACIÓN DE EMPLEADOS
# =====
empleados = []
roles = ['Ventas', 'Tecnico', 'F&I']
for i in range(1, 21):
    empleados.append({
        'empleado_id': i,
        'nombre_completo': fake.name(),
        'rol': random.choices(roles, weights=[50, 40, 10])[0] # Más vendedores y té
    })
df_empleados = pd.DataFrame(empleados)
vendedores_ids = df_empleados[df_empleados['rol'] == 'Ventas']['empleado_id'].tolist
tecnicos_ids = df_empleados[df_empleados['rol'] == 'Tecnico']['empleado_id'].tolist

# =====
# 3. GENERACIÓN DE CLIENTES
# =====
clientes = []
for i in range(1, 501):
    clientes.append({
        'cliente_id': i,
        'nombre': fake.name(),
        'email': fake.ascii_free_email(),
        'telefono': fake.phone_number(),
        'ciudad': fake.city(),
        'tipo_cliente': random.choices(['Regular', 'Premium'], weights=[80, 20])[0]
    })
df_clientes = pd.DataFrame(clientes)

# =====
# 4. GENERACIÓN DE VEHÍCULOS
# =====
marcas_modelos = {
    'Toyota': ['Corolla', 'RAV4', 'Hilux'],
    'Ford': ['Escape', 'F-150', 'Explorer'],
    'Honda': ['Civic', 'CR-V', 'Accord'],
    'Nissan': ['Sentra', 'Versa', 'Frontier']
}

vehiculos = []
vines_generados = []
for _ in range(300):
    vin = fake.bothify(text='?#?#?#?#?#?#?#?', letters='ABCDEFGHJKLMNPRSTUVWXYZ')
    marca = random.choice(list(marcas_modelos.keys()))
    modelo = random.choice(marcas_modelos[marca])
    anio = random.randint(2018, 2024)
    condicion = 'Nuevo' if anio >= 2023 else 'Seminuevo'
    fecha_ingreso = fake.date_between(start_date='-2y', end_date='today')

```

```

vehiculos.append({
    'vin': vin,
    'marca': marca,
    'modelo': modelo,
    'anio': anio,
    'condicion': condicion,
    'costo_compra': round(random.uniform(15000, 45000), 2),
    'estado_inventario': 'Disponible', # Se actualizará luego si se vende
    'fecha_ingreso': fecha_ingreso
})
vines_generados.append(vin)
df_vehiculos = pd.DataFrame(vehiculos)

# =====
# 5. GENERACIÓN DE LEADS & VENTAS (Logica de negocio)
# =====
leads = []
ventas = []
motivos_perdida = ['Precio alto', 'No aprobó crédito', 'Compró en competencia', 'So

lead_id_counter = 1
venta_id_counter = 1

for vin in vines_generados:
    vehiculo = df_vehiculos[df_vehiculos['vin'] == vin].iloc[0]
    fecha_ingreso = vehiculo['fecha_ingreso']

    # Cada vehículo genera entre 1 y 3 Leads
    num_leads = random.randint(1, 3)
    fue_vendido = False

    for _ in range(num_leads):
        if fue_vendido:
            break # Si ya se vendió, no genera más Leads

        cliente_id = random.randint(1, 500)
        vendedor_id = random.choice(vendedores_ids)
        # El lead se crea DESPUÉS del ingreso del vehículo
        fecha_creacion_lead = fecha_ingreso + timedelta(days=random.randint(1, 30))

        # Probabilidad de cierre: 30%
        estado_lead = random.choices(
            ['Nuevo', 'Test Drive', 'Negociacion', 'Cerrado', 'Perdido'],
            weights=[10, 20, 20, 30, 20]
        )[0]

        motivo = random.choice(motivos_perdida) if estado_lead == 'Perdido' else No

    leads.append({
        'lead_id': lead_id_counter,
        'cliente_id': cliente_id,
        'vehiculo_interes_vin': vin,
        'vendedor_id': vendedor_id,
        'fecha_creacion': fecha_creacion_lead,
        'estado_lead': estado_lead,
        'motivo_perdida': motivo
    })
    lead_id_counter += 1

```

```

    })

    if estado_lead == 'Cerrado':
        fue_vendido = True
        # Venta ocurre entre 1 y 15 días después del lead
        fecha_venta = fecha_creacion_lead + timedelta(days=random.randint(1, 15))
        precio_venta = vehiculo['costo_compra'] * random.uniform(1.05, 1.25) #

        ventas.append({
            'venta_id': venta_id_counter,
            'lead_id': lead_id_counter,
            'cliente_id': cliente_id,
            'vehiculo_vin': vin,
            'vendedor_id': vendedor_id,
            'fecha_venta': fecha_venta,
            'precio_venta': round(precio_venta, 2),
            'incluye_garantia_ext': random.choice([True, False]),
            'incluye_credito': random.choice([True, False])
        })
        # Actualizar estado de inventario
        df_vehiculos.loc[df_vehiculos['vin'] == vin, 'estado_inventario'] = 'Vendida'
        venta_id_counter += 1

    lead_id_counter += 1

df_leads = pd.DataFrame(leads)
df_ventas = pd.DataFrame(ventas)

# =====
# 6. GENERACIÓN DE ÓRDENES DE REPARACIÓN (Taller)
# =====
ordenes = []
or_id_counter = 1

# Solo entran al taller vehículos que ya fueron vendidos
for index, venta in df_ventas.iterrows():
    # Probabilidad del 60% de que vuelva al servicio al menos una vez
    if random.random() < 0.6:
        num_visitas = random.randint(1, 3)
        ultima_fecha = venta['fecha_venta']

        for _ in range(num_visitas):
            # Visita al taller ocurre después de la venta (ej. mantenimientos cada
            fecha_apertura = ultima_fecha + timedelta(days=random.randint(90, 180))
            tecnico_id = random.choice(tecnicos_ids)
            horas = round(random.uniform(1.0, 5.0), 2)

            ordenes.append({
                'or_id': or_id_counter,
                'cliente_id': venta['cliente_id'],
                'vehiculo_vin': venta['vehiculo_vin'],
                'tecnico_id': tecnico_id,
                'fecha_apertura': fecha_apertura,
                'horas_facturadas': horas,
                'costo_repuestos': round(random.uniform(50, 400), 2),
                'costo_mano_obra': round(horas * 60, 2) # $60 por hora
            })
            or_id_counter += 1

```

```

    })
    ultima_fecha = fecha_apertura
    or_id_counter += 1

df_ordenes = pd.DataFrame(ordenes)

# =====
# 7. CARGA DE DATOS A MYSQL (LOAD)
# =====
print("Cargando datos a la base de datos OLTP...")

# Importante: El orden importa debido a Las llaves foráneas (Foreign Keys)
df_empleados.to_sql('empleados', con=engine, if_exists='append', index=False)
df_clientes.to_sql('clientes', con=engine, if_exists='append', index=False)
df_vehiculos.to_sql('vehiculos', con=engine, if_exists='append', index=False)
df_leads.to_sql('leads', con=engine, if_exists='append', index=False)
df_ventas.to_sql('ventas', con=engine, if_exists='append', index=False)
df_ordenes.to_sql('ordenes_reparacion', con=engine, if_exists='append', index=False)

print("¡Proceso completado con éxito! El sistema transaccional está listo.")

```

Construcción del Data Warehouse OLAP

Después de generar el sistema transaccional, se construyó un Data Warehouse orientado al análisis empresarial.

Durante este proceso ETL se realizaron las siguientes etapas:

Extract

Se extrajeron los datos desde la base OLTP.

Transform

Los datos fueron transformados para construir:

- Dimensiones
- Tablas de hechos
- Indicadores calculados
- Métricas de negocio

Load

Finalmente, la información fue cargada en el sistema OLAP.

Dimensiones creadas

- dim_cliente
- dim_vehiculo
- dim_empleado
- dim_tiempo

Tablas de hechos

- fact_ventas
- fact_servicios
- fact_inventario

Este modelo permite realizar análisis históricos y generar indicadores estratégicos para la empresa.

```
In [ ]: import pandas as pd
from sqlalchemy import create_engine
from datetime import datetime

# =====
# 1. CONEXIONES A MYSQL
# =====
# Reemplaza con tus credenciales
engine_oltp = create_engine("mysql+pymysql://root:@localhost:3306/concesionaria_oltp")
engine_olap = create_engine("mysql+pymysql://root:@localhost:3306/concesionaria_olap")

print("Conexiones establecidas. Iniciando proceso ETL...")

# =====
# 2. EXTRACT (Extracción desde OLTP)
# =====
print("Extrayendo datos del OLTP...")
df_empleados = pd.read_sql("SELECT * FROM empleados", con=engine_oltp)
df_clientes = pd.read_sql("SELECT * FROM clientes", con=engine_oltp)
df_vehiculos = pd.read_sql("SELECT * FROM vehiculos", con=engine_oltp)
df_ventas = pd.read_sql("SELECT * FROM ventas", con=engine_oltp)
df_ordenes = pd.read_sql("SELECT * FROM ordenes_reparacion", con=engine_oltp)

# =====
# 3. TRANSFORM (Transformación)
# =====
print("Transformando datos para el modelo OLAP...")

# 3.1 Crear Dimensión Tiempo Dinámica
def generar_dim_tiempo(start_date, end_date):
    date_range = pd.date_range(start=start_date, end=end_date)
    dim_tiempo = pd.DataFrame({
        'fecha': date_range,
        'fecha_key': date_range.strftime('%Y%m%d').astype(int),
        'dia': date_range.day,
```

```

        'mes': date_range.month,
        'anio': date_range.year,
        'trimestre': date_range.quarter,
        'dia_semana': date_range.day_name() # En inglés, se puede mapear a español
    })
    return dim_tiempo

# Encontrar fecha mínima y máxima entre ventas, órdenes e ingresos
fechas_todas = pd.concat([df_ventas['fecha_venta'], df_ordenes['fecha_apertura'], d
fecha_min = fechas_todas.min()
fecha_max = fechas_todas.max() # 0 datetime.today().date()
dim_tiempo = generar_dim_tiempo(fecha_min, fecha_max)

# 3.2 Transformar Dimensiones Clientes, Vehículos, Empleados
dim_cliente = df_clientes[['cliente_id', 'nombre', 'ciudad', 'tipo_cliente']].renam
dim_vehiculo = df_vehiculos[['vin', 'marca', 'modelo', 'anio', 'condicion']].rename
dim_empleado = df_empleados[['empleado_id', 'nombre_completo', 'rol']].rename(column

# 3.3 Transformar Fact Ventas
fact_ventas = df_ventas.copy()
fact_ventas['fecha_key'] = pd.to_datetime(fact_ventas['fecha_venta']).dt.strftime('
# Unir con vehiculos para traer el costo y calcular el margen bruto
fact_ventas = fact_ventas.merge(df_vehiculos[['vin', 'costo_compra']], left_on='veh
fact_ventas['margen_bruto'] = fact_ventas['precio_venta'] - fact_ventas['costo_comp

fact_ventas = fact_ventas[[
    'venta_id', 'fecha_key', 'cliente_id', 'vehiculo_vin', 'vendedor_id',
    'precio_venta', 'costo_compra', 'margen_bruto', 'incluye_garantia_ext', 'incluy
]].rename(columns={
    'cliente_id': 'cliente_key',
    'vehiculo_vin': 'vehiculo_key',
    'vendedor_id': 'vendedor_key',
    'incluye_garantia_ext': 'incluye_garantia'
})

# 3.4 Transformar Fact Servicios (Taller)
fact_servicios = df_ordenes.copy()
fact_servicios['fecha_key'] = pd.to_datetime(fact_servicios['fecha_apertura']).dt.s
fact_servicios['ingreso_total'] = fact_servicios['costo_repuestos'] + fact_servicio

fact_servicios = fact_servicios[[
    'or_id', 'fecha_key', 'cliente_id', 'vehiculo_vin', 'tecnico_id', 'horas_factur
]].rename(columns={
    'cliente_id': 'cliente_key',
    'vehiculo_vin': 'vehiculo_key',
    'tecnico_id': 'tecnico_key'
})

# 3.5 Transformar Fact Inventario (Ageing)
fact_inventario = df_vehiculos[['vin', 'fecha_ingreso', 'costo_compra', 'estado_inv
fact_inventario['fecha_ingreso_key'] = pd.to_datetime(fact_inventario['fecha_ingres

# Buscar fecha de venta (si existe)
ventas_fechas = df_ventas[['vehiculo_vin', 'fecha_venta']].copy()
ventas_fechas['fecha_venta_key'] = pd.to_datetime(ventas_fechas['fecha_venta']).dt.

```

```

fact_inventario = fact_inventario.merge(ventas_fechas, left_on='vin', right_on='veh

# Calcular días en inventario
fecha_hoy = pd.to_datetime(datetime.today().date())
fact_inventario['fecha_ingreso'] = pd.to_datetime(fact_inventario['fecha_ingreso'])
fact_inventario['fecha_venta'] = pd.to_datetime(fact_inventario['fecha_venta'])

# Si se vendió, los días son fecha_venta - fecha_ingreso. Si no, hoy - fecha_ingres
fact_inventario['dias_en_inventario'] = fact_inventario.apply(
    lambda row: (row['fecha_venta'] - row['fecha_ingreso']).days if pd.notnull(row[
        axis=1
    ])

fact_inventario = fact_inventario[[
    'vin', 'fecha_ingreso_key', 'fecha_venta_key', 'costo_compra', 'estado_inventar
]].rename(columns={'vin': 'vehiculo_key', 'estado_inventario': 'estado_actual'})
# Convertir NaN a None (o un valor int por defecto) para MySQL en fecha_venta_key
fact_inventario['fecha_venta_key'] = fact_inventario['fecha_venta_key'].astype('Int

# =====
# 4. LOAD (Carga al OLAP)
# =====
print("Cargando datos al Data Warehouse (OLAP)...")

# Cargar Dimensiones primero (por Las llaves foráneas)
dim_tiempo.to_sql('dim_tiempo', con=engine_olap, if_exists='append', index=False)
dim_cliente.to_sql('dim_cliente', con=engine_olap, if_exists='append', index=False)
dim_vehiculo.to_sql('dim_vehiculo', con=engine_olap, if_exists='append', index=False)
dim_empleado.to_sql('dim_empleado', con=engine_olap, if_exists='append', index=False)

# Cargar Hechos
fact_ventas.to_sql('fact_ventas', con=engine_olap, if_exists='append', index=False)
fact_servicios.to_sql('fact_servicios', con=engine_olap, if_exists='append', index=
fact_inventario.to_sql('fact_inventario', con=engine_olap, if_exists='append', inde

print("¡Proceso ETL completado con éxito! El Data Warehouse está listo para ser ana

```

Inteligencia de Negocio y KPIs

Una vez construido el Data Warehouse, se desarrollaron distintos análisis de Business Intelligence para apoyar a la gerencia en la toma de decisiones.

Los análisis implementados fueron:

- Segmentación de clientes mediante Machine Learning
- Análisis de inventario crítico
- Rentabilidad de vendedores
- Costos financieros por vehículos estancados
- Indicadores de taller y postventa

Estos análisis permiten detectar oportunidades de mejora y optimizar la rentabilidad de la concesionaria.

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sqlalchemy import create_engine
import warnings
warnings.filterwarnings('ignore')

# Configuración visual para los gráficos
sns.set_theme(style="whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)

# =====
# 1. CONEXIÓN AL DATA WAREHOUSE (OLAP)
# =====
# Reemplaza con tus credenciales
engine = create_engine("mysql+pymysql://root:2006@localhost:3306/concesionaria_olap")
print("Conectado al OLAP. Extrayendo inteligencia de negocio...")

# =====
# 2. MODELO DE CLUSTERING (K-MEANS): SEGMENTACIÓN DE CLIENTES
# =====
print("\n--- Ejecutando Segmentación de Clientes ---")

# Consulta SQL analítica: Agregamos el comportamiento del cliente en Ventas y Taller
query_clientes = """
SELECT
    c.cliente_key,
    c.nombre,
    COUNT(DISTINCT v.venta_id) as autos_comprados,
    COALESCE(SUM(v.precio_venta), 0) as gastado_autos,
    COALESCE(AVG(v.margen_bruto / v.precio_venta), 0) as porcentaje_margen,
    COUNT(DISTINCT s.or_id) as visitas_taller,
    COALESCE(SUM(s.ingreso_total), 0) as gastado_taller
FROM dim_cliente c
LEFT JOIN fact_ventas v ON c.cliente_key = v.cliente_key
LEFT JOIN fact_servicios s ON c.cliente_key = s.cliente_key
GROUP BY c.cliente_key, c.nombre
HAVING autos_comprados > 0 OR visitas_taller > 0;
"""

df_clientes = pd.read_sql(query_clientes, con=engine)

# Selección de features para el K-Means
features = ['gastado_autos', 'porcentaje_margen', 'visitas_taller', 'gastado_taller']
X = df_clientes[features]

# Escalar los datos (vital para K-Means)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```

# Entrenar el modelo
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df_clientes['cluster'] = kmeans.fit_predict(X_scaled)

# Asignar nombres comerciales a los clusters basándonos en los centroides
# (Lógica simplificada: mayor margen y autos = Premium, bajo margen = Buscadores, s
cluster_means = df_clientes.groupby('cluster')[features].mean()

def nombrar_cluster(row):
    if row['gastado_autos'] == 0 and row['visitas_taller'] > 0:
        return 'Solo Servicio'
    elif row['porcentaje_margen'] < cluster_means['porcentaje_margen'].mean():
        return 'Buscadores de Ofertas'
    else:
        return 'Premium / Fieles'

df_clientes['Segmento'] = df_clientes.apply(nombrar_cluster, axis=1)

print("Distribución de Clientes:")
print(df_clientes['Segmento'].value_counts())

# Gráfico 1: Scatter plot de Clientes
plt.figure()
sns.scatterplot(data=df_clientes, x='gastado_autos', y='gastado_taller', hue='Segmento')
plt.title("Segmentación de Clientes (LTV Autos vs Taller)", fontsize=14, fontweight='bold')
plt.xlabel("Monto Gastado en Vehículos ($)")
plt.ylabel("Monto Gastado en Taller ($)")
plt.savefig("grafico_segmentacion_clientes.png")
print("-> Gráfico guardado como 'grafico_segmentacion_clientes.png'")

# =====
# 3. REPORTE PRESCRIPTIVO: DINERO ESTANCADO Y COSTO DE OPORTUNIDAD
# =====
print("\n--- Ejecutando Análisis de Inventario y Floorplan ---")

# Extraer inventario disponible
query_inventario = """
SELECT
    i.vehiculo_key,
    v.marca,
    v.modelo,
    v.condicion,
    i.costo_compra,
    i.dias_en_inventario
FROM fact_inventario i
JOIN dim_vehiculo v ON i.vehiculo_key = v.vehiculo_key
WHERE i.estado_actual = 'Disponible'
"""

df_inventario = pd.read_sql(query_inventario, con=engine)

# Variables Financieras Asumidas
TASA_INTERES_ANUAL = 0.08 # 8% anual de Floorplan
DEPRECIACION_MENSUAL = 0.015 # 1.5% mensual de pérdida de valor comercial

# Cálculo del Daily Holding Cost (Costo Financiero Diario)
tasa_diaria_total = (TASA_INTERES_ANUAL / 365) + (DEPRECIACION_MENSUAL / 30)

```

```

df_inventario['costo_diario'] = df_inventario['costo_compra'] * tasa_diaria_total
df_inventario['perdida_acumulada'] = df_inventario['costo_diario'] * df_inventario[

# Filtrar autos críticos (> 90 días)
aged_inventory = df_inventario[df_inventario['dias_en_inventario'] > 90].copy()

# Algoritmo de Propuesta de Descuento (Price Markdown)
# Lógica: Es mejor descontar el 80% de lo que ya hemos perdido en costo financiero
# antes de seguir perdiendo dinero mañana.
aged_inventory['descuento_sugerido'] = aged_inventory['perdida_acumulada'] * 0.8
aged_inventory['nuevo_precio_minimo'] = aged_inventory['costo_compra'] - aged_inven

print(f"\nAlerta: Tienes {len(aged_inventory)} vehículos con más de 90 días.")
print(f"Pérdida financiera acumulada total: ${aged_inventory['perdida_acumulada'].s

if not aged_inventory.empty:
    print("\nTop 5 Vehículos Críticos a Descontar:")
    print(aged_inventory[['marca', 'modelo', 'dias_en_inventario', 'costo_compra',

# Gráfico 2: Distribución de Ageing y Pérdida
plt.figure()
sns.histplot(data=df_inventario, x='dias_en_inventario', weights='perdida_acumulada
plt.axvline(90, color='black', linestyle='--', label='Límite Crítico (90 días)')
plt.title("Costo Financiero Estancado vs Edad del Inventario", fontsize=14, fontwei
plt.xlabel("Días en Inventario (Ageing)")
plt.ylabel("Pérdida Acumulada ($ USD)")
plt.legend()
plt.savefig("grafico_dinero_estancado.png")
print("-> Gráfico guardado como 'grafico_dinero_estancado.png'")

# =====
# 4. CUADRO DE MANDO: EFICIENCIA DE VENDEDORES (F&I UPSELL)
# =====
print("\n--- Analizando Rentabilidad por Vendedor (Front-End vs Back-End) ---")

query_ventas = """
SELECT
    e.nombre as vendedor,
    COUNT(v.venta_id) as total_unidades,
    SUM(v.margen_bruto) as utilidad_metal,
    SUM(CASE WHEN v.incluye_garantia = 1 THEN 1 ELSE 0 END) as ventas_garantia,
    SUM(CASE WHEN v.incluye_credito = 1 THEN 1 ELSE 0 END) as ventas_credito
FROM fact_ventas v
JOIN dim_empleado e ON v.vendedor_key = e.empleado_key
GROUP BY e.nombre
"""

df_rendimiento = pd.read_sql(query_ventas, con=engine)

# Calcular penetración
df_rendimiento['penetracion_back_end'] = ((df_rendimiento['ventas_garantia'] + df_r

# Gráfico 3: Rentabilidad vs Penetración de Seguros/Créditos
plt.figure()
sns.barplot(data=df_rendimiento.sort_values('utilidad_metal', ascending=False).head
            x='utilidad_metal', y='vendedor', color='steelblue', label='Utilidad Me

```

```
# Eje secundario para la penetración
ax2 = plt.twinx()
sns.scatterplot(data=df_rendimiento.sort_values('utilidad_metal', ascending=False),
                x='penetracion_back_end', y='vendedor', color='orange', s=150, label='vendedor')

plt.title("Top Vendedores: Utilidad Bruta vs Eficiencia en F&I (Seguros/Créditos)",
          fontweight='bold')
plt.savefig("grafico_eficiencia_vendedores.png")
print("-> Gráfico guardado como 'grafico_eficiencia_vendedores.png'")

print("\n¡Análisis completo! Los reportes están listos para la junta con el Gerente")
```

Dashboard Ejecutivo Interactivo

Finalmente, se desarrolló un dashboard interactivo utilizando Streamlit.

El objetivo principal fue presentar la información de forma visual y comprensible para la gerencia.

El dashboard incluye:

- Indicadores de ventas
- Rentabilidad
- Segmentación de clientes
- Inventario crítico
- KPIs financieros
- Rendimiento de taller
- Métricas comerciales

Esta herramienta permite monitorear el negocio en tiempo real y facilitar la toma de decisiones estratégicas.

```
In [ ]: import streamlit as st
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from sqlalchemy import create_engine
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# Configuración de la página
st.set_page_config(page_title="Executive Dealer Insights", layout="wide", page_icon="🚗")

# Persona del CTO
st.title("🚗 Automotive Strategy Dashboard")
st.markdown("""
*CTO & Lead Data Scientist Insights* | Analítica Prescriptiva para la Toma de Decis
---
""")
```

```

# =====
# CONEXIÓN A BASES DE DATOS (OLAP)
# =====
@st.cache_resource
def get_connection():
    # Asegúrate de usar tus credenciales
    return create_engine("mysql+pymysql://root:@localhost:3306/concesionaria_olap")

engine = get_connection()

# =====
# CARGA DE DATOS (TRANSFORMACIÓN ANALÍTICA)
# =====
@st.cache_data
def load_data():
    df_ventas = pd.read_sql("SELECT * FROM fact_ventas v JOIN dim_vehiculo veh ON v

    # FIX: Added v.condicion to the SELECT statement
    df_inventario = pd.read_sql("SELECT i.*, v.marca, v.modelo, v.condicion FROM fa

    df_servicios = pd.read_sql("SELECT * FROM fact_servicios", engine)
    df_clientes = pd.read_sql("""
        SELECT c.*,
            (SELECT SUM(precio_venta) FROM fact_ventas v WHERE v.cliente_key = c.client
            (SELECT SUM(ingreso_total) FROM fact_servicios s WHERE s.cliente_key = c.cl
            (SELECT COUNT(*) FROM fact_servicios s WHERE s.cliente_key = c.cliente_key)
        FROM dim_cliente c""", engine)
    return df_ventas, df_inventario, df_servicios, df_clientes

df_ventas, df_inventario, df_servicios, df_clientes = load_data()

# =====
# SIDEBAR - FILTROS ESTRATÉGICOS
# =====
st.sidebar.header("Filtros Globales")
marca_filter = st.sidebar.multiselect("Marca", options=df_ventas['marca'].unique(),
condicion_filter = st.sidebar.radio("Condición", options=['Todos', 'Nuevo', 'Seminu

# Aplicar filtros
df_v_filtered = df_ventas[df_ventas['marca'].isin(marca_filter)]
if condicion_filter != 'Todos':
    df_v_filtered = df_v_filtered[df_v_filtered['condicion'] == condicion_filter]

# =====
# LAYOUT DE PESTAÑAS
# =====
tab1, tab2, tab3, tab4 = st.tabs(["📊 Desempeño Comercial", "💰 Inventario Crítico

# --- PESTAÑA 1: VENTAS Y F&I ---
with tab1:
    st.header("KPIs de Ventas y F&I")
    col1, col2, col3, col4 = st.columns(4)

    total_revenue = df_v_filtered['precio_venta'].sum()
    total_units = len(df_v_filtered)
    avg_margin = df_v_filtered['margen_bruto'].mean()

```

```

f_and_i_penetration = (df_v_filtered['incluye_credito'].sum() / total_units) *

col1.metric("Ingresos Totales", f"${total_revenue:,.2f}")
col2.metric("Unidades Entregadas", f"{total_units}")
col3.metric("Margen Promedio", f"${avg_margin:,.2f}")
col4.metric("Penetración Crédito", f"{f_and_i_penetration:.1f}%")

# Gráfico de Ventas Mensuales
df_ventas_mes = df_v_filtered.copy()
df_ventas_mes['mes'] = df_ventas_mes['fecha_key'].apply(lambda x: str(x)[4:6])
fig_ventas = px.line(df_ventas_mes.groupby('mes')['precio_venta'].sum().reset_index(
    x='mes', y='precio_venta', title="Evolución Mensual de Ven
st.plotly_chart(fig_ventas, use_container_width=True)

# --- PESTAÑA 2: INVENTARIO CRÍTICO (AGEING) ---
with tab2:
    st.header("Análisis de Dinero Estancado")
    st.warning("Vehículos con más de 90 días generan un costo de oportunidad crítico")

    # Cálculos Financieros
    df_inv_disp = df_inventario[df_inventario['estado_actual'] == 'Disponible']
    df_inv_disp['costo_holding_acumulado'] = df_inv_disp['costo_compra'] * 0.0005 *

    col_inv1, col_inv2 = st.columns([2, 1])

    with col_inv1:
        fig_ageing = px.histogram(df_inv_disp, x="dias_en_inventario", color="condi
            marginal="box", title="Distribución de Días en I
        st.plotly_chart(fig_ageing, use_container_width=True)

    with col_inv2:
        st.subheader("Unidades Críticas")
        aged_critical = df_inv_disp[df_inv_disp['dias_en_inventario'] > 90][['marca
        st.dataframe(aged_critical.sort_values('dias_en_inventario', ascending=False

    # Propuesta de Markdown
    st.subheader("Prescripción: Propuesta de Descuento para Rotación")
    if not aged_critical.empty:
        aged_critical['Descuento Sugerido (80% Holding)'] = aged_critical['costo_co
        st.table(aged_critical)
    else:
        st.success("No hay unidades con envejecimiento crítico.")

# --- PESTAÑA 3: MACHINE LEARNING (CLUSTERING) ---
with tab3:
    st.header("Segmentación de Clientes via K-Means")

    # Preparar datos para Clustering
    df_c = df_clientes.fillna(0)
    X = df_c[['gasto_autos', 'gasto_taller', 'visitas_taller']]
    X_scaled = StandardScaler().fit_transform(X)

    kmeans = KMeans(n_clusters=3, random_state=42).fit(X_scaled)
    df_c['Cluster'] = kmeans.labels_

    # Mapear nombres

```

```

cluster_map = {0: "Cazadores de Ofertas", 1: "Premium / Fieles", 2: "Solo Servi
df_c['Segmento'] = df_c['Cluster'].map(cluster_map)

fig_cluster = px.scatter(df_c, x="gasto_autos", y="gasto_taller", color="Segmen
                        size="visitas_taller", hover_name="nombre",
                        title="Clusters de Clientes: Valor de Vida (LTV)")
st.plotly_chart(fig_cluster, use_container_width=True)

st.info("💡 *Estrategia del CTO:* Los clientes en el segmento 'Premium' deben r

# -----
# CÓDIGO DE LA NUEVA PESTAÑA 4
# -----
with tab4:
    st.header("Operaciones Fijas y Eficiencia Financiera")
    st.markdown("Indicadores de rentabilidad estructural y rendimiento de taller.")

    # --- SIMULACIÓN DE DATOS (Si tu ETL aún no los tiene, usa esto como fallback)
    # Asumimos gastos fijos mensuales de la concesionaria ($150,000 USD)
    gastos_fijos_concesionaria = 150000

    # Cálculos para Absorción
    utilidad_taller = df_servicios['ingreso_total'].sum() # En la vida real, sería
    indice_absorcion = (utilidad_taller / gastos_fijos_concesionaria) * 100

    # Cálculos para PRU (Per Retail Unit) en F&I
    # Asumimos que cada garantía deja $500 y cada crédito deja $800 de comisión par
    df_v_filtered['profit_garantia'] = df_v_filtered['incluye_garantia'] * 500
    df_v_filtered['profit_credito'] = df_v_filtered['incluye_credito'] * 800
    total_f_and_i_profit = df_v_filtered['profit_garantia'].sum() + df_v_filtered['
    pru_financiero = total_f_and_i_profit / len(df_v_filtered) if len(df_v_filtered

    # Cálculos de Taller (ELR)
    # Asumimos horas facturadas = horas en df_servicios. (Usamos un mock si 'horas_
    if 'horas_reales_trabajadas' not in df_servicios.columns:
        import numpy as np
        df_servicios['horas_reales_trabajadas'] = df_servicios['horas_facturadas']

    eficiencia_tecnicos = (df_servicios['horas_facturadas'].sum() / df_servicios['h
    effective_labor_rate = df_servicios['ingreso_total'].sum() / df_servicios['hora

    # --- RENDERIZADO DE KPIs ---
    colA, colB, colC, colD = st.columns(4)
    colA.metric("PRU (F&I Profit por Unidad)", f"${pru_financiero:,.2f}", "+12% vs
    colB.metric("Effective Labor Rate (ELR)", f"${effective_labor_rate:,.2f} / hr",
    colC.metric("Eficiencia de Taller", f"{eficiencia_tecnicos:.1f}%", "+2.5% vs Be

    # Costo de Adquisición (Mockup simple asumiendo CAC promedio)
    cac_promedio = 350 # Costo de marketing promedio por venta
    margen_promedio = df_v_filtered['margen_bruto'].mean()
    roi_marketing = ((margen_promedio - cac_promedio) / cac_promedio) * 100
    colD.metric("ROI de Marketing (GP vs CAC)", f"{roi_marketing:.1f}%", f"CAC: ${c

    st.markdown("---")

    # --- GRÁFICOS AVANZADOS ---

```

```

col_graph1, col_graph2 = st.columns([1, 1])

with col_graph1:
    # Velocímetro de Absorción (Gauge Chart)
    fig_absorcion = go.Figure(go.Indicator(
        mode = "gauge+number",
        value = indice_absorcion,
        domain = {'x': [0, 1], 'y': [0, 1]},
        title = {'text': "Índice de Absorción (Service Absorption Rate)", 'font':
            {'size': 14, 'color': 'red'}},
        number = {'suffix': "%"},
        gauge = {
            'axis': {'range': [0, 120]},
            'bar': {'color': "darkblue"},
            'steps': [
                {'range': [0, 60], 'color': "lightcoral"},
                {'range': [60, 85], 'color': "gold"},
                {'range': [85, 120], 'color': "lightgreen"}],
            'threshold': {
                'line': {'color': "black", 'width': 4},
                'thickness': 0.75,
                'value': 100}
        }
    ))
    fig_absorcion.update_layout(height=350)
    st.plotly_chart(fig_absorcion, use_container_width=True)
    st.caption("🎯 Objetivo: 100%. Si alcanzamos 100%, el taller paga todas las")

with col_graph2:
    # Gráfico de Dispersión: Eficiencia vs ELR por Técnico
    # Simulamos datos agregados por técnico para el ejemplo
    df_tech = df_servicios.groupby('tecnico_key').agg(
        horas_facturadas=('horas_facturadas', 'sum'),
        horas_reales=('horas_reales_trabajadas', 'sum'),
        ingreso_total=('ingreso_total', 'sum')
    ).reset_index()

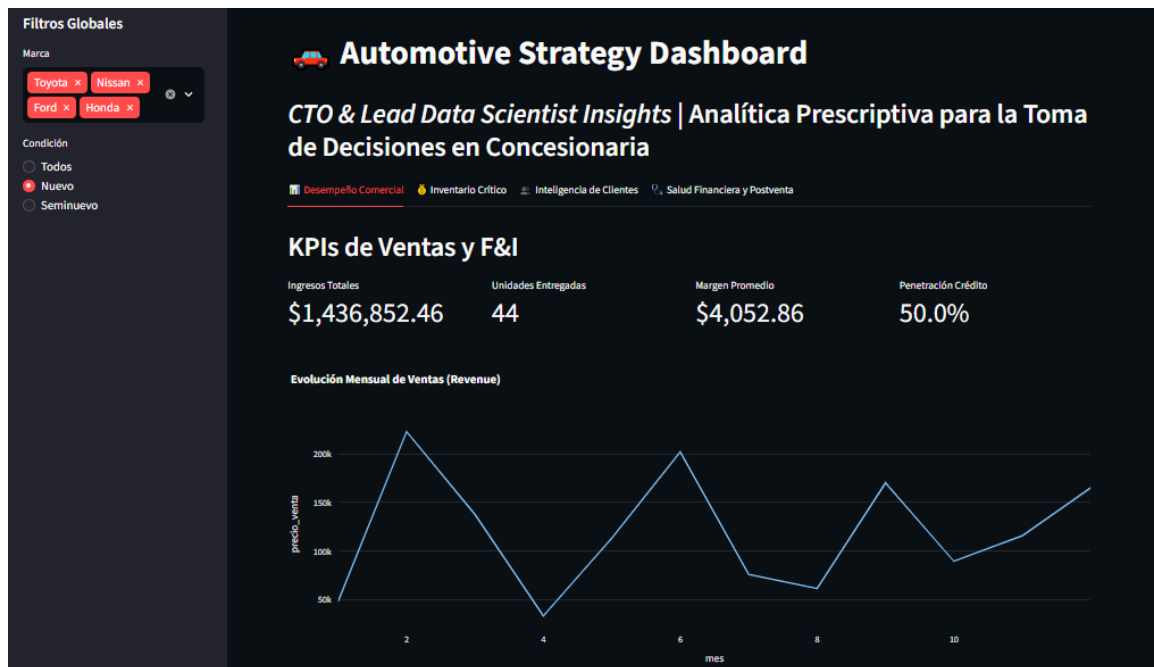
    df_tech['Eficiencia (%)'] = (df_tech['horas_facturadas'] / df_tech['horas_reales']) * 100
    df_tech['ELR ($)'] = df_tech['ingreso_total'] / df_tech['horas_facturadas']
    df_tech['tecnico_key'] = df_tech['tecnico_key'].astype(str) # Convertir a string

    fig_tech = px.scatter(df_tech, x="Eficiencia (%)", y="ELR ($)",
        color="tecnico_key", size="horas_facturadas",
        title="Cuadrante de Productividad por Técnico",
        labels={"tecnico_key": "ID Técnico"})

    # Añadir Líneas de Benchmark
    fig_tech.add_hline(y=100, line_dash="dash", line_color="red", annotation_text="100% Eficiencia")
    fig_tech.add_vline(x=100, line_dash="dash", line_color="green", annotation_text="100% ELR")

    fig_tech.update_layout(height=350)
    st.plotly_chart(fig_tech, use_container_width=True)

```



Resultados Obtenidos

La implementación del sistema Business Intelligence permitió transformar datos operativos en información estratégica para la concesionaria.

Entre los principales resultados obtenidos destacan:

- Identificación de clientes premium y clientes orientados únicamente a servicios.
- Detección de vehículos con exceso de tiempo en inventario.
- Análisis de vendedores con mejor rendimiento comercial.
- Evaluación de ingresos generados por servicios de taller.
- Visualización de métricas financieras en tiempo real.

El sistema desarrollado facilita la comprensión del comportamiento del negocio y permite tomar decisiones más rápidas y fundamentadas.

Además, la integración entre OLTP y OLAP demuestra cómo los datos operacionales pueden convertirse en inteligencia empresarial útil para la organización.

Conclusiones

1. El uso de Business Intelligence permitió convertir grandes volúmenes de datos en información útil para apoyar la toma de decisiones gerenciales.
2. La implementación del sistema OLTP facilitó el registro organizado de las operaciones diarias de la concesionaria, mientras que el sistema OLAP permitió analizar el

comportamiento del negocio desde una perspectiva estratégica.

3. El proceso ETL permitió integrar y transformar la información operacional en un Data Warehouse preparado para consultas analíticas y generación de indicadores.
4. Los dashboards y KPIs desarrollados ayudan a identificar oportunidades de mejora relacionadas con ventas, clientes, inventario y rentabilidad.
5. La segmentación de clientes mediante Machine Learning permitió identificar perfiles de consumidores que pueden ser utilizados para campañas comerciales más efectivas.
6. El análisis del inventario crítico permitió detectar vehículos con alto tiempo de permanencia, ayudando a reducir pérdidas financieras y mejorar la rotación de inventario.
7. El proyecto demuestra cómo la analítica de datos puede aportar valor real a las empresas y apoyar procesos de planificación y crecimiento empresarial.